PPO, Explained from the Mathematics to PyTorch

An undergraduate guide to Proximal Policy Optimization, its equations, and its relationship to an ordinary neural network

The central idea

PPO is not a special neural-network layer. It is an actor–critic training procedure built around an ordinary differentiable network. Its distinctive ingredients are stochastic data collection, advantage estimation, an old/new policy ratio, and a clipped objective that limits the incentive for overly large favorable changes.

Based on

Schulman, Wolski, Dhariwal, Radford, and Klimov,

Proximal Policy Optimization Algorithms, arXiv:1707.06347v2 (2017).

Re-typeset as a LaTeX-native study guide with vector mathematics and figures.

August 2026

Contents

1 Technical summary: PPO is a training rule, not a layer **1**

2 One mental model for the entire algorithm **2**

3 Notation: the vocabulary behind the equations **3**

4 From the policy gradient to PPO **3**
4.1 Equation (1): the policy-gradient direction . . . 3
4.2 Equation (2): an objective autograd can differentiate . . . 4
4.3 Equations (3)–(5): the trust-region idea PPO simplifies . . . 4

5 The defining PPO idea: the clipped probability ratio **4**
5.1 Equation (6): compare the new and old policies . . . 4
5.2 Equation (7): PPO's clipped surrogate objective . . . 5
5.3 The clipping plateau depends on the advantage sign . . . 6
5.4 Four numerical cases make the minimum concrete . . . 6
5.5 Equation (8): the adaptive-KL alternative . . . 7

6 The complete actor–critic objective **7**
6.1 Equation (9): actor, critic, and entropy in one expression . . . 7

7 Returns, bootstrapping, and Generalized Advantage Estimation **7**
7.1 Equation (10): a finite-horizon bootstrapped advantage . . . 7
7.2 Equations (11)–(12): Generalized Advantage Estimation . . . 8

8 Algorithm 1: how the pieces run in time **8**

9 PPO versus an ordinary PyTorch multilayer perceptron **9**
9.1 The forward computation is still an ordinary network . . . 9
9.2 What the actor output means . . . 10

10 The mathematics mapped directly into PyTorch **10**
10.1 A symbol-to-code bridge . . . 10
10.2 A minimal actor–critic network . . . 10
10.3 Equation (7) translated almost literally into code . . . 11

11 A tiny end-to-end numerical example **12**

12 What the equations do—and do not—guarantee **12**

13 A practical way to check your understanding **12**

14 Good next questions **13**

1 Technical summary: PPO is a training rule, not a layer

Proximal Policy Optimization (PPO) is an actor–critic reinforcement-learning algorithm. It gathers experience using a stochastic policy, estimates which sampled actions were better or worse than expected, and updates the policy using a clipped objective that discourages a large change from the policy that collected the data [1].

- A network such as Linear → activation → Linear → output is simply a parameterized differentiable function.
- PPO can use exactly that architecture as its **actor**; it interprets the output as parameters of an action distribution.
- PPO usually trains a **critic** that predicts the expected discounted return from a state.
- What makes the full system PPO is the **rollout loop, advantage estimate, old/new probability ratio, clipped loss, and repeated minibatch updates**—not a special layer type.

The conceptual spine is

$$\boxed{\text{ordinary network} \rightarrow \text{stochastic policy} \rightarrow \text{sampled rollout} \rightarrow \widehat{A}_t \rightarrow r_t(\theta) \rightarrow \text{clipped update.}}$$

The most important expression is Equation (7). It allows one on-policy batch to be used for several optimization epochs while removing the extra reward for moving a sampled action's probability too far in a favorable direction. This is a practical safety rail, not a proof that every update will be small.

2 One mental model for the entire algorithm

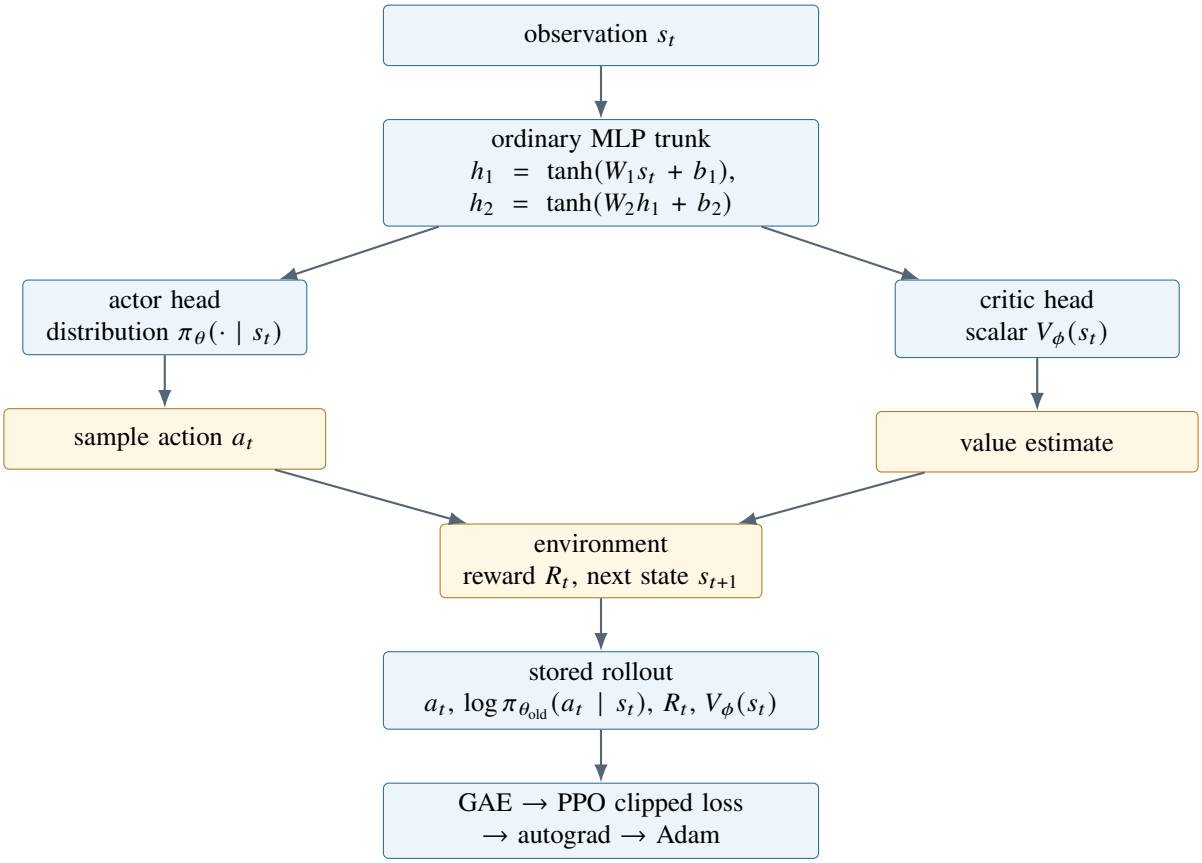


Figure 1: The neural-network computation is familiar PyTorch. PPO supplies the data-collection and loss-construction machinery around it.

3 Notation: the vocabulary behind the equations

Symbol	Meaning
s_t	State or observation at time t; for a robot, this might include joint angles, joint velocities, body orientation, and commanded velocity.
a_t	Action sampled at time t; for a robot, perhaps desired joint positions, velocities, or torques.
R_t	Scalar reward observed after the action. This guide uses capital R_t to avoid confusing reward with PPO's probability ratio.
$\pi_\theta(a \mid s)$	Policy probability mass (discrete actions) or probability density (continuous actions); θ denotes actor parameters.
$V_\phi(s)$	Critic estimate of expected future discounted reward from state s; ϕ denotes critic parameters.
$Q^\pi(s,a)$	Expected future discounted reward after taking action a in state s, then following policy π.
$A^\pi(s,a)$	Advantage, $Q^\pi(s,a) - V^\pi(s)$. Positive means better than expected; negative means worse.
$\gamma \in [0,1]$	Discount factor. A reward k steps away receives weight γ^k.
$\lambda \in [0,1]$	GAE parameter controlling the bias–variance tradeoff.
$\widehat{\mathbb{E}}_t[\cdot]$	Empirical average over the finite state–action samples in the current batch.

A notation trap in the original paper

Section 3 uses $r_t(\theta)$ for the *new/old policy probability ratio*, whereas Equations (10)–(12) use r_t for reward. In this guide, $r_t(\theta)$ always means the ratio and R_t always means reward.

4 From the policy gradient to PPO

4.1 Equation (1): the policy-gradient direction

$$\widehat{g} = \widehat{\mathbb{E}}_t\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, \widehat{A}_t\right]. \tag{1}$$

Read the expression one sample at a time:

1. $\log \pi_\theta(a_t \mid s_t)$ asks: *how much log-probability does the current policy assign to the action that was actually sampled?*
2. ∇_θ asks how a small parameter change would change that log-probability.
3. $\widehat{A}_t$ provides the sign and strength of the update. If the sampled action was better than expected, $\widehat{A}_t > 0$; if worse, $\widehat{A}_t < 0$.
4. $\widehat{\mathbb{E}}_t$ averages these sample-level suggestions over the batch.

The logarithm appears through the score-function identity

$$\nabla_\theta \pi_\theta(a \mid s) = \pi_\theta(a \mid s)\, \nabla_\theta \log \pi_\theta(a \mid s).$$

It converts the derivative of an expectation over sampled actions into an expectation that can be estimated from trajectories. Log-probabilities are also numerically convenient because products of probabilities become sums of logs.

4.2 Equation (2): an objective autograd can differentiate

$$L^{\mathrm{PG}}(\theta) = \widehat{\mathbb{E}}_t\left[\log \pi_\theta(a_t \mid s_t)\, \widehat{A}_t\right]. \tag{2}$$

If the advantage estimates are treated as fixed numbers during the actor update, differentiating Equation (2) produces Equation (1). The paper writes the objective for *maximization*; PyTorch optimizers normally minimize, so an implementation uses $-L^{\mathrm{PG}}$ as its policy loss.

Why not optimize Equation (2) for many epochs? The samples came from an older policy. After several updates, the current policy may be far from the data-collecting policy, yet the same fixed positive advantages can keep pushing selected probabilities upward. PPO's central job is to make limited reuse of the batch less dangerous.

4.3 Equations (3)–(5): the trust-region idea PPO simplifies

Trust Region Policy Optimization (TRPO) measures change in *distribution space*, not weight space. Its surrogate problem can be written as

$$\underset{\theta}{\text{maximize}} \quad \widehat{\mathbb{E}}_t\left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}\, \widehat{A}_t\right], \tag{3}$$

$$\text{subject to} \quad \widehat{\mathbb{E}}_t\left[\mathrm{D}_{\mathrm{KL}}\big(\pi_{\theta_{\text{old}}}(\cdot \mid s_t) \,\|\, \pi_\theta(\cdot \mid s_t)\big)\right] \le \delta. \tag{4}$$

The Kullback–Leibler (KL) divergence measures how distinguishable the two action distributions are. The threshold δ is a step-size budget in policy space. This matters because a small weight change can occasionally cause a large policy change, while different weight vectors may still define similar policies.

A penalty form replaces the hard constraint with

$$\underset{\theta}{\text{maximize}} \quad \widehat{\mathbb{E}}_t\left[r_t(\theta)\widehat{A}_t - \beta\, \mathrm{D}_{\mathrm{KL}}\big(\pi_{\theta_{\text{old}}}(\cdot \mid s_t) \,\|\, \pi_\theta(\cdot \mid s_t)\big)\right]. \tag{5}$$

A fixed β is awkward: too small permits destructive changes; too large prevents useful learning. TRPO solves a constrained approximation. PPO seeks a simpler first-order objective compatible with ordinary minibatches and optimizers.

5 The defining PPO idea: the clipped probability ratio

5.1 Equation (6): compare the new and old policies

For the particular action stored in the rollout, define

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}.$$

The conservative policy-iteration surrogate is

$$L^{\mathrm{CPI}}(\theta) = \widehat{\mathbb{E}}_t\left[r_t(\theta)\widehat{A}_t\right]. \tag{6}$$

At the beginning of an update, $\theta = \theta_{\text{old}}$, so $r_t(\theta) = 1$. Later:

- $r_t(\theta) = 1.20$: the stored action is 20% more likely, or has 20% greater density, under the new policy;
- $r_t(\theta) = 0.75$: it has 25% less probability or density;
- $r_t(\theta) = 1$: its probability or density did not change.

For numerical stability, implementations subtract log-probabilities before exponentiating:

```
ratio = torch.exp(new_log_prob - old_log_prob)
```

For a multidimensional continuous action, each log-probability should be the *joint* log-density, commonly obtained by summing over action dimensions.

5.2 Equation (7): PPO's clipped surrogate objective

$$L^{\mathrm{CLIP}}(\theta) = \widehat{\mathbb{E}}_t\left[\min\left(r_t(\theta)\widehat{A}_t, \mathrm{clip}\big(r_t(\theta), 1-\epsilon, 1+\epsilon\big)\widehat{A}_t\right)\right]. \tag{7}$$

With $\epsilon = 0.2$, the clipped ratio lies in $[0.8, 1.2]$. The minimum is the subtle part: it selects the more pessimistic of the ordinary and clipped terms.

- If $\widehat{A}_t > 0$, increasing the sampled action's probability is favorable, but the objective stops improving once $r_t(\theta) > 1+\epsilon$.
- If $\widehat{A}_t < 0$, decreasing the sampled action's probability is favorable, but the objective stops improving once $r_t(\theta) < 1-\epsilon$.
- If the policy moves in the *wrong* direction, the worse unclipped value remains. Clipping does not erase the penalty for a harmful move.

Therefore PPO is not simply "clamp the ratio and multiply." The clipping is one-sided and depends on the sign of the advantage.

5.3 The clipping plateau depends on the advantage sign

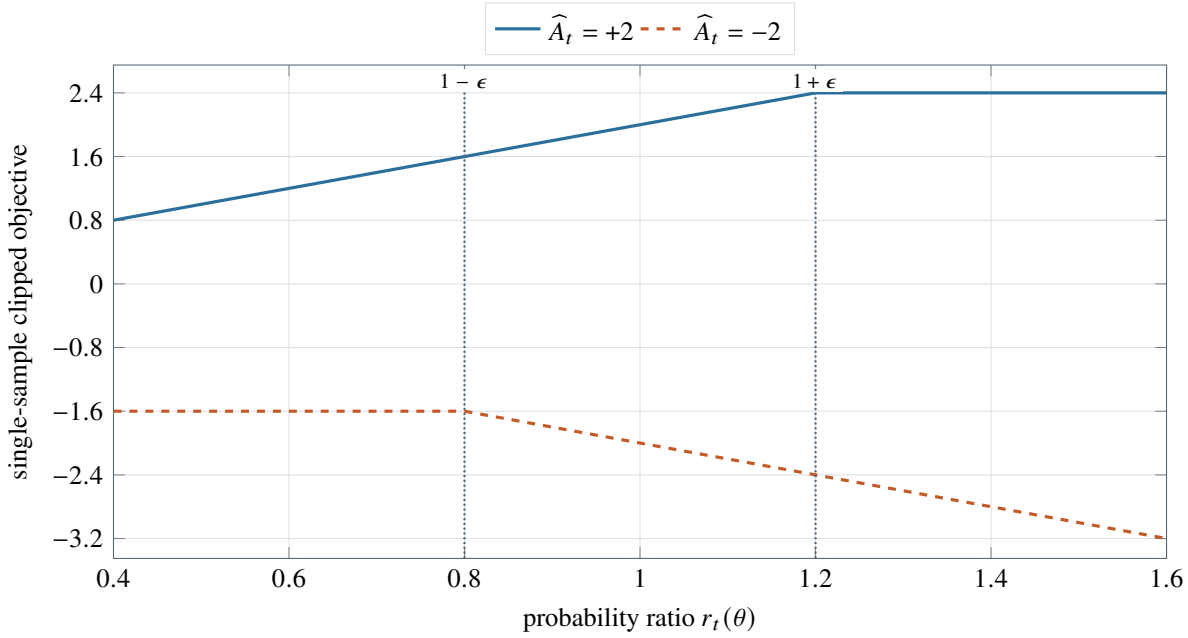


Figure 2: Equation (7) for $\epsilon = 0.2$. The positive-advantage term plateaus above 1.2; the negative-advantage term plateaus below 0.8. Line style as well as color distinguishes the two cases.

Figure 2 holds the advantage at +2 or −2 and varies only the ratio. A flat segment means that this particular sample no longer asks the optimizer to move farther in a favorable direction. These are not learning curves; they are two sample-level objective contributions. A real update averages many terms with different states, actions, and advantages.

5.4 Four numerical cases make the minimum concrete

Using $\epsilon = 0.2$:

$$\begin{aligned}
&\widehat{A}_t = +2,\ r_t = 1.4: && \min(1.4 \times 2,\ 1.2 \times 2) = 2.4, && \text{favorable change capped;}\\
&\widehat{A}_t = +2,\ r_t = 0.6: && \min(0.6 \times 2,\ 0.8 \times 2) = 1.2, && \text{full unfavorable penalty;}\\
&\widehat{A}_t = -2,\ r_t = 0.6: && \min(0.6 \times -2,\ 0.8 \times -2) = -1.6, && \text{favorable change capped;}\\
&\widehat{A}_t = -2,\ r_t = 1.4: && \min(1.4 \times -2,\ 1.2 \times -2) = -2.8, && \text{full unfavorable penalty.}
\end{aligned}$$

Away from the two nondifferentiable boundaries, the sample-level derivative with respect to the ratio is

$$\frac{\partial \ell^{\mathrm{CLIP}}(r, A)}{\partial r} = \begin{cases} 0, & A > 0 \text{ and } r > 1+\epsilon, \\ 0, & A < 0 \text{ and } r < 1-\epsilon, \\ A, & \text{otherwise.} \end{cases}$$

Other samples still contribute gradients, and shared network parameters couple many states together. Consequently the final policy may still move beyond the interval for some samples.

Important limitation

Clipped PPO discourages large favorable changes but does not impose TRPO's hard KL constraint. Monitoring approximate KL divergence and stopping an epoch early is a common additional safeguard; it is not part of Equation (7) itself.

5.5 Equation (8): the adaptive-KL alternative

$$L^{\mathrm{KLPEN}}(\theta) = \widehat{\mathbb{E}}_t\left[r_t(\theta)\widehat{A}_t - \beta\, \mathrm{D}_{\mathrm{KL}}\left(\pi_{\theta_{\mathrm{old}}}(\cdot \mid s_t) \,\|\, \pi_\theta(\cdot \mid s_t)\right)\right]. \tag{8}$$

After an update, the mean KL divergence d is compared with a target d_{target}. The paper halves β when $d < d_{\mathrm{target}}/1.5$ and doubles it when $d > 1.5 d_{\mathrm{target}}$. This is feedback control: strengthen the penalty when policy changes are too large and relax it when they are too small. The paper's experiments favored the clipped objective over this adaptive-penalty variant.

6 The complete actor–critic objective

6.1 Equation (9): actor, critic, and entropy in one expression

$$L^{\mathrm{CLIP+VF+S}}(\theta) = \widehat{\mathbb{E}}_t\left[L_t^{\mathrm{CLIP}}(\theta) - c_1 L_t^{\mathrm{VF}}(\theta) + c_2\, \mathcal{H}[\pi_\theta(\cdot \mid s_t)]\right], \tag{9}$$

with a squared value-function error such as

$$L_t^{\mathrm{VF}}(\phi) = \left(V_\phi(s_t) - \widehat{V}_t^{\mathrm{target}}\right)^2.$$

The paper maximizes Equation (9):

- L_t^{CLIP} improves the actor without rewarding an excessively large favorable ratio change;
- L_t^{VF} is the critic's squared prediction error, so it enters with a minus sign in a maximization objective;
- $\mathcal{H}[\pi_\theta]$ is policy entropy, which rewards a less concentrated distribution and usually promotes exploration;
- c_1 and c_2 balance the scales of the three terms.

Because PyTorch minimizes, a practical loss has the sign pattern

$$\mathcal{L}_{\mathrm{min}} = -\overline{L^{\mathrm{CLIP}}} + c_1 \overline{L^{\mathrm{VF}}} - c_2 \overline{\mathcal{H}}.$$

If actor and critic share a trunk, gradients from both heads reach the shared layers. If they use separate networks, their losses can be optimized separately.

7 Returns, bootstrapping, and Generalized Advantage Estimation

7.1 Equation (10): a finite-horizon bootstrapped advantage

For a trajectory segment ending at time T, define

$$\widehat{A}_t = -V_\phi(s_t) + \sum_{l=0}^{T-t-1} \gamma^l R_{t+l} + \gamma^{T-t} V_\phi(s_T). \tag{10}$$

The sum contains rewards that actually occurred inside the segment. The final term $\gamma^{T-t}V_\phi(s_T)$ estimates rewards beyond an artificial rollout cutoff; this is **bootstrapping**. Subtracting $V_\phi(s_t)$ converts the estimated return into an advantage.

If s_T is a true terminal state, the bootstrap value is normally zero. If the segment ended only because the rollout horizon was reached, setting it to zero would incorrectly pretend that the future has no value.

7.2 Equations (11)–(12): Generalized Advantage Estimation

Generalized Advantage Estimation (GAE) combines a sequence of one-step temporal-difference residuals:

$$\widehat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}. \tag{11}$$

Here each residual is

$$\delta_t = R_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t). \tag{12}$$

The TD residual is a local surprise signal:

$$\delta_t = \underbrace{R_t}_{\text{reward received}} + \underbrace{\gamma V_\phi(s_{t+1})}_{\text{estimated next value}} - \underbrace{V_\phi(s_t)}_{\text{value predicted before acting}}.$$

- $\lambda = 0$: use only δ_t. This tends to lower variance but depends more strongly on critic accuracy and is usually more biased.
- $\lambda \approx 1$: use a longer chain of residuals. This tends to reduce bootstrapping bias but increase variance.
- The factor $(\gamma\lambda)^l$ geometrically downweights distant residuals.

For $\lambda = 1$, the intermediate value terms telescope:

$$\begin{aligned}\sum_{l=0}^{T-t-1} \gamma^l \delta_{t+l} &= \sum_{l=0}^{T-t-1} \gamma^l \big(R_{t+l} + \gamma V_\phi(s_{t+l+1}) - V_\phi(s_{t+l})\big) \\ &= \sum_{l=0}^{T-t-1} \gamma^l R_{t+l} + \gamma^{T-t} V_\phi(s_T) - V_\phi(s_t),\end{aligned}$$

which is exactly Equation (10). This cancellation is the mathematical bridge between finite-horizon returns and GAE.

8 Algorithm 1: how the pieces run in time

1. Freeze a snapshot of the actor, denoted $\pi_{\theta_{\text{old}}}$.
2. Run N actors for T steps and store observations, actions, rewards, termination flags, value estimates, and $\log \pi_{\theta_{\text{old}}}(a_t \mid s_t)$.
3. Work backward through each trajectory to compute return targets and GAE advantages.
4. Treat stored actions, old log-probabilities, returns, and advantages as fixed training data during the update.
5. Shuffle the NT samples into minibatches.

6. For K epochs, recompute the new log-probability of each *stored* action, form $r_t(\theta)$, and optimize the clipped actor loss together with critic and entropy terms.
7. Replace the old policy snapshot with the updated policy, retire the batch, and collect fresh on-policy experience.

The paper's MuJoCo benchmark used $T = 2048$, Adam step size 3×10^{-4}, 10 epochs, minibatches of 64, $\gamma = 0.99$, and $\lambda = 0.95$. These are experiment settings, not mathematical constants or universal defaults.

9 PPO versus an ordinary PyTorch multilayer perceptron

9.1 The forward computation is still an ordinary network

For a two-hidden-layer actor–critic trunk,

$$\begin{aligned} h_1 &= \tanh(W_1 s_t + b_1), \\ h_2 &= \tanh(W_2 h_1 + b_2), \\ \mu_\theta(s_t) &= W_\mu h_2 + b_\mu, \\ V_\phi(s_t) &= W_V h_2 + b_V. \end{aligned}$$

These are ordinary affine maps and nonlinear activations. PPO does not change their algebra; it changes what the outputs mean, where training data come from, and which loss produces the gradients.

Table 1: The relationship between a normal MLP and the complete PPO system.

Aspect	Ordinary supervised MLP	PPO actor–critic system
Core computation	Linear layers, nonlinear activations, differentiable outputs	The same layers and activations may be used
Input	Generic feature vector x	State or observation s_t
Output meaning	Class logits or regression prediction	Parameters of $\pi_\theta(\cdot \mid s_t)$, plus value $V_\phi(s_t)$
Targets	Fixed labels y from a dataset	Rewards, return targets, advantages, and old log-probabilities generated by rollouts
Data source	Usually a fixed external dataset	Interaction between the current policy and the environment
Loss	Classification or regression loss	Clipped actor objective + value error - entropy bonus
Data reuse	Often many epochs over the same examples	Limited epochs because on-policy data become stale as the actor changes
Optimizer	SGD, Adam, or another gradient method	The same ordinary optimizers and autograd machinery

The architecture answers *what differentiable function is computed?* PPO answers *how are targets generated, and how is that function updated using on-policy experience without rewarding an excessive favorable change?*

9.2 What the actor output means

For **discrete actions**, the final linear layer emits logits $z_1, \dots, z_m$. A categorical policy uses

$$\pi_\theta(a = j \mid s) = \frac{e^{z_j(s)}}{\sum_{k=1}^{m} e^{z_k(s)}}.$$

Sampling chooses an action, and $\log \pi_\theta(a_t \mid s_t)$ supplies the quantity used in Equations (1), (2), (6), and (7).

For **continuous actions**, the actor often produces a Gaussian mean and a learned or state-dependent standard deviation:

$$a_t \sim \mathcal{N}\Big(\mu_\theta(s_t), \operatorname{diag}\Big(\sigma_\theta^2(s_t)\Big)\Big), \qquad \sigma_\theta(s) = \exp(\log \sigma_\theta(s)) .$$

The paper's continuous-control model used two 64-unit hidden layers with tanh activations and a Gaussian action distribution with variable standard deviations. That is a normal MLP architecture; PPO is the method used to train it.

For bounded robot actions, implementations often apply a transformation such as tanh to Gaussian samples. The log-density must then include the change-of-variables correction. Blindly clipping a sample after drawing it does not automatically give the correct density for the executed action.

10 The mathematics mapped directly into PyTorch

10.1 A symbol-to-code bridge

Mathematical symbols are used in the derivation; descriptive identifiers are used in executable code. The two are related as follows:

Table 2: Translation between scholarly notation and the implementation.

Mathematics	PyTorch identifier	Meaning
s_t	`obs`	current observations
a_t	`action`	actions stored during the rollout
$\log \pi_\theta(a_t \mid s_t)$	`new_log_prob`	log-probability under the updated actor
$\log \pi_{\theta_{\text{old}}}(a_t \mid s_t)$	`old_log_prob`	frozen rollout log-probability
$r_t(\theta)$	`ratio`	new/old probability ratio
$\widehat{A}_t$	`advantage`	GAE or another advantage estimate
ϵ	`eps`	clipping half-width
$\widehat{V}_t^{\text{target}}$	`return_target`	critic regression target

10.2 A minimal actor–critic network

```
import torch
import torch.nn as nn
from torch.distributions import Independent, Normal

class ActorCritic(nn.Module):
    def __init__(self, obs_dim, act_dim):
        super().__init__()
        self.trunk = nn.Sequential(
            nn.Linear(obs_dim, 64), nn.Tanh(),
```

```
            nn.Linear(64, 64), nn.Tanh(),
        )
        self.actor_mean = nn.Linear(64, act_dim)
        self.critic = nn.Linear(64, 1)
        self.log_std = nn.Parameter(torch.zeros(act_dim))

    def forward(self, obs):
        h = self.trunk(obs)
        mean = self.actor_mean(h)
        value = self.critic(h).squeeze(-1)
        std = self.log_std.exp().expand_as(mean)
        policy = Independent(Normal(mean, std), 1)
        return policy, value
```

`nn.Linear` computes an affine map, and `nn.Tanh` supplies the nonlinearity. `Independent(..., 1)` treats the action dimensions as one joint event, so `log_prob(action)` returns one joint log-density per observation [2, 3].

10.3 Equation (7) translated almost literally into code

```
# Fixed tensors saved or computed from the rollout:
# obs, action, old_log_prob, advantage, return_target

policy, value = model(obs)
new_log_prob = policy.log_prob(action)
ratio = torch.exp(new_log_prob - old_log_prob)

eps = 0.2
surr_unclipped = ratio * advantage
surr_clipped = (
    torch.clamp(ratio, 1.0 - eps, 1.0 + eps) * advantage
)
policy_loss = -torch.minimum(
    surr_unclipped, surr_clipped
).mean()

value_loss = 0.5 * (value - return_target).pow(2).mean()
entropy = policy.entropy().mean()

c1, c2 = 1.0, 0.01
loss = policy_loss + c1 * value_loss - c2 * entropy

optimizer.zero_grad()
loss.backward()
optimizer.step()
```

The mapping is direct:

- subtracting old from new log-probability and applying `exp` produces Equation (6)'s ratio;
- `minimum(unclipped, clipped)` implements Equation (7);
- the leading minus converts maximization into minimization;
- value loss and entropy supply the other terms from Equation (9);
- `backward()` differentiates through the same linear and activation layers used in ordinary PyTorch training.

Stored old log-probabilities, advantages, and return targets must be detached from the update graph. PPO does not resample the action inside the loss; it evaluates the new policy's probability of the *action stored in the rollout*.

11 A tiny end-to-end numerical example

Suppose the old policy assigned a forward torque command density 0.10, while the updated policy assigns density 0.13. Then

$$r_t(\theta) = \frac{0.13}{0.10} = 1.3.$$

Assume the action worked better than expected, so $\widehat{A}_t = +4$, and use $\epsilon = 0.2$. Equation (7) gives

$$\begin{aligned} \text{unclipped term} &= 1.3 \times 4 = 5.2, \\ \text{clipped ratio} &= \text{clip}(1.3, 0.8, 1.2) = 1.2, \\ \text{clipped term} &= 1.2 \times 4 = 4.8, \\ \ell_t^{\text{CLIP}} &= \min(5.2, 4.8) = 4.8. \end{aligned}$$

The action is still encouraged, but this sample provides no extra credit for increasing its density beyond the 20% boundary. Another state–action sample, or a sample with negative advantage, may activate a different boundary. The batch average combines these local votes before Adam changes the shared weights.

12 What the equations do—and do not—guarantee

- **Clipping is a heuristic trust-region surrogate.** It does not guarantee a hard KL bound or monotonic policy improvement.
- **The critic can be wrong.** Biased value estimates produce biased advantages. The pair (γ, λ) trades bias against variance rather than eliminating both.
- **On-policy data are expensive.** Multiple epochs improve sample reuse, but too many epochs make a rollout increasingly stale.
- **The three loss scales interact.** Learning rate, clipping range, advantage normalization, value coefficient, entropy coefficient, batch size, epoch count, and network initialization all affect training.
- **The paper is an algorithm description, not a complete modern implementation specification.** Observation normalization, reward scaling, value clipping, gradient clipping, termination handling, action transforms, and KL early stopping require deliberate choices.
- **For a real robot, a correct objective is not a hardware safety system.** Train and stress-test in simulation, constrain actions, handle communication loss, and retain independent torque, position, velocity, and shutdown protections.

13 A practical way to check your understanding

You understand the core of PPO if you can answer these without memorization:

1. Why does a positive advantage make gradient ascent increase the sampled action's log-probability?
2. Why must the ratio denominator use the frozen data-collecting policy?
3. For negative advantage, why is the useful clipping boundary $1 - \epsilon$, not $1 + \epsilon$?
4. Why does PyTorch code negate the clipped objective?
5. How does the critic reduce variance without directly selecting the action?

6. Why is a standard MLP compatible with PPO even though the MLP itself has no PPO-specific layer?

A strong next exercise is to implement the four numerical clipping cases as tensors, call `backward()`, and inspect the derivative with respect to the ratio. The gradient is zero only in favorable beyond-boundary cases selected by the minimum.

14 Good next questions

- How should Gaussian actions be squashed to actuator limits while preserving the correct transformed log-probability?
- What observation and reward normalization are appropriate for a quadruped?
- Should actor and critic share a trunk or use separate networks?
- How should true terminal states differ from time-limit truncations in GAE?
- Which diagnostics—approximate KL, clip fraction, entropy, explained variance, and gradient norm—best reveal a failing PPO run?

These questions concern implementation details. The core remains an ordinary network inside a specialized reinforcement-learning data and optimization loop.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
https://arxiv.org/abs/1707.06347

[2] PyTorch Contributors, "`torch.nn.Linear`," PyTorch documentation.
https://docs.pytorch.org/docs/stable/generated/torch.nn.Linear.html

[3] PyTorch Contributors, "Probability Distributions," PyTorch documentation.
https://docs.pytorch.org/docs/stable/distributions.html